

Clase 060 — Ataques criptográficos: padding oracle y timing

Parte: **2 — Criptografía aplicada** · Fuente: *Serious Cryptography* (Aumasson) y *Cryptography Engineering* (Ferguson/Schneier/Kohno) ⌚ Duración estimada: **120 min** · Nivel: **Avanzado**

🎯 Objetivo

Entender que la mayoría de los sistemas criptográficos no se rompen atacando el algoritmo, sino explotando su **implementación**. El alumno estudiará dos familias emblemáticas: el ataque de padding oracle (descifrar sin la clave abusando de mensajes de error de padding en CBC) y los ataques de canal lateral por tiempo (deducir secretos midiendo cuánto tarda una operación). Ambos se practican **solo** en un servicio de laboratorio propio.

⚠️ **Nota ética:** estos ataques se ejecutan exclusivamente contra un oráculo/servidor montado por ti en tu laboratorio. Aplicarlos a sistemas de terceros sin autorización explícita es ilegal.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** cómo un padding oracle permite descifrar CBC byte a byte.
- 2. **Montar** un oráculo vulnerable de laboratorio y atacarlo de forma controlada.
- 3. **Describir** ataques de timing sobre comparaciones y operaciones cripto.
- 4. **Aplicar** mitigaciones: AEAD, verificación en tiempo constante, mensajes de error uniformes.
- 5. **Reconocer** por qué "fallar cerrado y en tiempo constante" es esencial.

📖 Temas

#	Tema	Por qué importa
1	Ataques a la implementación	Donde ocurren las brechas reales
2	Padding PKCS#7 y su verificación	Origen del oráculo
3	Padding oracle paso a paso	Descifrado sin clave
4	Ataques de timing	Canal lateral temporal
5	Comparación en tiempo constante	Mitigación clave
6	AEAD como defensa	Elimina el oráculo
7	Casos reales (POODLE, Lucky13)	Impacto histórico

Definiciones y características

- **Canal lateral (side-channel):** fuga de información por medios ajenos al algoritmo (tiempo, energía, errores). Característica: rompe cripto teóricamente segura.
- **Padding oracle:** servicio que revela (por error o comportamiento) si el padding de un texto descifrado es válido, permitiendo recuperar el plano.
- **PKCS#7:** esquema de relleno en CBC; su verificación distinta de otros errores crea el oráculo.
- **Ataque de timing:** se infiere un secreto midiendo diferencias de tiempo de ejecución (p. ej. comparación con salida temprana).
- **Tiempo constante:** código cuyo tiempo no depende de datos secretos; imprescindible en comparaciones y operaciones con claves.
- **Fallar cerrado:** rechazar sin distinguir causas ni entregar datos parciales.
- **Lucky 13 / POODLE:** ataques reales que explotaron padding y timing en TLS/CBC.

Herramientas y preparación

```
pip install cryptography flask requests
```

Monta el oráculo en `localhost`. No apuntes las herramientas a ningún host externo.

Laboratorio guiado

1. **Monta un oráculo vulnerable** (laboratorio propio). Un pequeño servicio Flask descifra AES-CBC y responde "padding OK" o "padding inválido". Ese comportamiento distinguible es el oráculo.
2. **Ataque de padding oracle.** Implementa el ataque clásico: para cada bloque, manipula el bloque previo byte a byte hasta que el oráculo indique padding válido; despeja el "intermediate value" y recupera el texto plano. Recupera un mensaje completo sin conocer la clave.
3. **Mitígalo con AEAD.** Reescribe el servicio con AES-GCM: ahora cualquier manipulación falla con `InvalidTag` de forma uniforme y el ataque deja de funcionar.
4. **Timing en comparación de tokens.** Implementa una comparación byte a byte con salida temprana y mide (con muchas repeticiones) que un token con más prefijo correcto tarda un poco más. Sustitúyela por `hmac.compare_digest` y comprueba que la diferencia desaparece.
5. **Documenta las mitigaciones:** AEAD, errores uniformes, comparación constante, y no exponer distinciones de fallo.

Ejercicios

1. Explica por qué CBC + verificación de padding revela información.
2. Recupera un bloque con el oráculo de tu laboratorio y describe cada paso.
3. Demuestra empíricamente una diferencia de timing en una comparación ingenua.
4. Reescribe el servicio con AEAD y verifica que el ataque falla.
5. Investiga cómo Lucky 13 explotó timing en el MAC de TLS-CBC.
6. Propón cómo unificar mensajes de error para no filtrar la causa.

Reto verificable

Toma un oráculo de padding de laboratorio y recupera un texto plano completo sin la clave; luego aplica una mitigación (migrar a AEAD) y demuestra que el mismo ataque ya no recupera nada. **Criterio de aceptación:** entregas el texto plano recuperado en la versión vulnerable y muestras que, tras migrar a AES-GCM, el atacante solo obtiene `InvalidTag` sin información útil.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Mensajes de error distintos para padding vs MAC	Crea un oráculo; unifica errores
Comparación de tags con salida temprana	Timing; usa <code>compare_digest</code>
CBC sin autenticación	Vulnerable a padding oracle; usa AEAD
Logs que revelan la causa del fallo	Fuga; registra sin distinguir para el cliente
Descifrar antes de verificar integridad	Procesas datos manipulables; verifica primero

Preguntas frecuentes

 **¿Basta con "esconder" los mensajes de error?** No; el timing u otros canales siguen filtrando. La solución real es AEAD y tiempo constante, no ocultar síntomas.

 **¿Por qué AEAD elimina el padding oracle?** Porque verifica el tag antes de tocar el padding y falla de forma uniforme, sin revelar validez de relleno.

 **¿Los ataques de timing son realistas por red?** Sí; con suficientes mediciones y estadística se explotan incluso a través de la red (Lucky 13 lo demostró).

Referencias

- Vaudenay, "Security Flaws Induced by CBC Padding" (padding oracle original).
- AlFardan & Paterson, "Lucky Thirteen" — <http://www.isg.rhul.ac.uk/tls/Lucky13.html>
- Aumasson, *Serious Cryptography*, cap. 4 y 9.
- OWASP, "Padding Oracle Attack".

Siguiente clase

Clase 061 - Introduccion al criptoanálisis